

Explicacion del codigo - ProjectForge PMI

Proposito

ProjectForge PMI es una aplicacion PHP + MySQL para gestionar el documento Plan de Gestion de Calidad. Permite registrar usuarios, aprobarlos, crear planes, editar informacion, cargar logos y generar una vista imprimible.

Capas del sistema

public/

Contiene los archivos que el navegador puede abrir directamente. Cada archivo representa una pantalla o accion:

- `login.php`: muestra el formulario de inicio de sesion.
- `register.php`: permite solicitar una cuenta.
- `users.php`: panel administrativo para aprobar usuarios.
- `index.php`: listado de planes.
- `form.php`: formulario principal del plan.
- `save.php`: guarda creaciones y ediciones.
- `view.php`: vista imprimible del documento.

app/

Contiene la logica que no debe estar duplicada en cada pantalla:

- `auth.php`: usuarios, roles, login, registro y aprobacion.
- `security.php`: CSRF, sesion segura y cabeceras HTTP.
- `repository.php`: CRUD del plan y sus secciones.
- `logo_upload.php`: subida, validacion y conversion de logos.
- `Database.php`: conexion PDO.
- `helpers.php`: utilidades pequenas para HTML, redireccion y validacion.
- `template.php`: valores por defecto de la plantilla EGPR230.

database/

Contiene los scripts SQL:

inicial.

- `plan_calidad_schema.sql`: crea todas las tablas.
- `plan_calidad_seed.sql`: crea datos iniciales, registros ejemplo y el admin

La entrega final deja solo los archivos necesarios para crear una instalacion nueva desde cero.

tests/

Contiene pruebas smoke:

- `smoke_auth.php`: valida registro, aprobacion, login y deshabilitacion.
- `smoke_repository.php`: valida CRUD del plan.
- `smoke_logo_resize.php`: valida conversion de logos.

Flujo de autenticacion

1. El usuario entra a una ruta protegida.
2. La pagina llama `require_login($pdo)`.
3. Si no hay usuario en sesion, la app redirige a `login.php`.
4. En login, `authenticate_user()` valida correo, password y estado.
5. Si el usuario esta active, se guarda `$_SESSION['user_id']`.
6. Si el usuario esta pending o disabled, se bloquea el acceso.
7. Si `must_change_password = 1`, se redirige a `change_password.php`.

Los administradores se validan con `require_admin($pdo)`.

Flujo de registro y aprobacion

1. `register.php` llama `register_user()`.
2. Se valida nombre, correo y contrasena.
3. La contrasena se guarda con `password_hash()`.
4. El usuario queda con rol user y estado pending.
5. Un admin entra a `users.php`.
6. `user_status.php` llama `update_user_admin_fields()`.
7. Si el estado cambia a active, el usuario ya puede entrar.

La app evita que el ultimo administrador activo se deshabilite a si mismo.

Rate limit de login

Cada intento de login se registra en `login_attempts`. Si un correo o IP supera 5 intentos fallidos en 15 minutos, el sistema bloquea temporalmente nuevos intentos. Esto reduce ataques de fuerza bruta contra el inicio de sesion.

Flujo de guardado de plan

1. `save.php` exige POST, sesion activa y CSRF valido.
2. `process_logo_uploads()` procesa los logos.
3. `create_or_update_plan()` limpia datos y valida campos.
4. Se abre una transaccion de base de datos.
5. Se inserta o actualiza `quality_plans`.
6. Las tablas hijas se reemplazan con funciones `replace_*`.
7. Se confirma la transaccion.
8. La app redirige a `view.php`.

Este diseno evita planes parcialmente guardados.

Seguridad contra SQL Injection

Las consultas usan PDO preparado:

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE email = ? LIMIT 1');  
$stmt->execute([$email]);
```

El usuario no se concatena dentro del SQL. Eso reduce el riesgo de SQL Injection.

Seguridad CSRF

Cada formulario POST incluye:

```
<?= csrf_input() ?>
```

Cada endpoint que modifica datos valida:

```
verify_csrf_token($_POST['_csrf'] ?? null);
```

Asi se evita que un sitio externo fuerce acciones desde el navegador del usuario.

Seguridad de archivos

Los logos pasan por `app/logo_upload.php`:

1. Se valida que el archivo venga de una subida real.
2. Se revisa el peso maximo.
3. Se confirma que sea imagen con `getimagesize()`.
4. Se acepta solo PNG, JPG o WebP.
5. Se convierte a WebP.
6. Se guarda con nombre aleatorio.
7. La ruta guardada solo puede ser `uploads/logos/{hash}.webp`.

La app no guarda URLs externas ni nombres originales.

Modelo de datos principal

`users` guarda cuentas y aprobaciones.

`quality_plans` guarda datos generales del documento.

Las tablas hijas guardan secciones repetibles:

- `quality_plan_versions`
- `quality_baselines`
- `process_improvement_steps`
- `quality_activity_matrix`
- `quality_roles`
- `quality_organization_items`
- `quality_normative_documents`

Modularidad

El proyecto separa las entradas web, la logica reutilizable, los SQL y las pruebas para que se puedan agregar nuevas paginas sin tocar toda la aplicacion.

Cuando se agregue un modulo nuevo, el patron recomendado es:

- Pantallas y acciones HTTP en `public/`.
- Reglas de negocio y persistencia en `app/`.
- Scripts de base de datos en `database/`.
- Pruebas smoke en `tests/`.

La guía completa esta en docs/modularidad.md.

Puntos de extension

Para agregar una API, se puede reutilizar app/auth.php y app/repository.php.

Para agregar mas plantillas PMI, conviene crear nuevos archivos similares a app/template.php.

Para agregar permisos mas finos, se pueden crear funciones can_*() en app/auth.php y llamarlas desde las paginas protegidas.

Para agregar paginas o modulos completos, seguir la guía docs/modularidad.md. El patron recomendado es crear una pantalla en public/, colocar la logica reutilizable en app/, proteger rutas con require_login() o require_admin(), y agregar una prueba smoke del flujo.

Checklist de produccion

- Cambiar password del admin inicial.
- Activar HTTPS.
- Configurar usuario de base de datos con permisos limitados.
- Exponer solo public/.
- Mantener backups de base de datos y logos.
- Ejecutar smoke tests antes de publicar cambios.